

Project Part 2 – ETL Pipeline and Serverless Function

1. Requirements

Develop an end-to-end ETL (Extract–Transform–Load) pipeline that ingests raw data, processes it into a harmonized form, and stores the results in cloud storage. The pipeline must trigger a serverless function whenever new data is uploaded.

1.1 Functional Requirements

- The raw files should be in CSV format, each containing multiple records from multiple devices (identified by a device_id column).
- Process the data with an ETL service.
- Deploy a serverless function that is triggered on new harmonized data.
- The function must send a notification in a Microsoft Teams Channel indicating whether the file was processed successfully or failed validation.
- Store processed output in cloud storage as structured files.
- Each run must produce one file per device. For each device, generate:
 - a latest/device-`<id>`.json file (overwritten each run), and
 - a by-timestamp/`<timestamp>/device-<id>`.json file (kept for history).
- The output should represent a simple, real-world scenario (e.g., energy usage, sales, IoT sensors). The JSON must include:
 - a unique identifier of the data source/device (e.g., device_id),
 - a generation timestamp,
 - a time window,
 - a list of records with a few relevant fields (e.g., timestamp, kWh, location),
 - (optional: a summary section with important aggregated values (e.g. total kWh))
- Provide a sample input file and demonstrate the pipeline output.

1.2 Implementation Technologies

- ETL Services: Azure Data Factory / AWS Glue / GCP Dataflow
- Serverless Functions: Azure Functions / AWS Lambda / GCP Cloud Functions

- Programming: Python (preferred for data transformation), Pandas/NumPy optional

1.3 Non-Functional Requirements

- Automated execution (trigger-based, not manual).
- Clear documentation of pipeline steps.
- Cloud-native storage (Blob/S3/GCS).

2. Deliverables

- Solution description document containing:
 - a) Conceptual architecture of the ETL pipeline.
 - b) UML Deployment diagram showing pipeline + function + storage.
 - c) README file with build and execution steps.
- Source files hosted on GitHub repository with the exact format:
 - CC2025_Group_LastName_FirstName_Project_2

Include:

- Serverless function code
- Pipeline definition (JSON, YAML, or Terraform/CDK if used)
- Documentation and sample input/output files

3. Evaluation

3.1 Project Related Basic Questions

During evaluation you may be asked about:

- ETL vs ELT concepts
- Cloud storage buckets and event triggers
- Serverless architecture (cold start, scaling, limits)
- Data transformation basics (schemas, JSON, CSV, Parquet)
- Python data libraries (Pandas, NumPy)
- Workflow orchestration principles

3.2 Grading

Points	Requirements
5 p	Minimum to pass: functional pipeline with ETL service + serverless function that processes data.
2 p	Correct integration between storage → ETL → serverless function (including notification to MS Teams).
1 p	Documentation with architecture diagram and clear steps.
1 p	The repository includes sample input and generated output for at least two different devices.
1 p	Code quality and repository structure (organized folders, proper commits).

Total: 10 points

Bonus (up to +1 p): If the entire pipeline and function deployment are automated using Infrastructure-as-Code (Terraform or CloudFormation).

4. Bibliography

1. Microsoft Learn – [Azure Data Factory](#)
2. AWS Docs – [AWS Glue](#)
3. Google Cloud – [Dataflow Documentation](#)
4. AWS Docs – [AWS Lambda](#)
5. Pandas Documentation – pandas.pydata.org